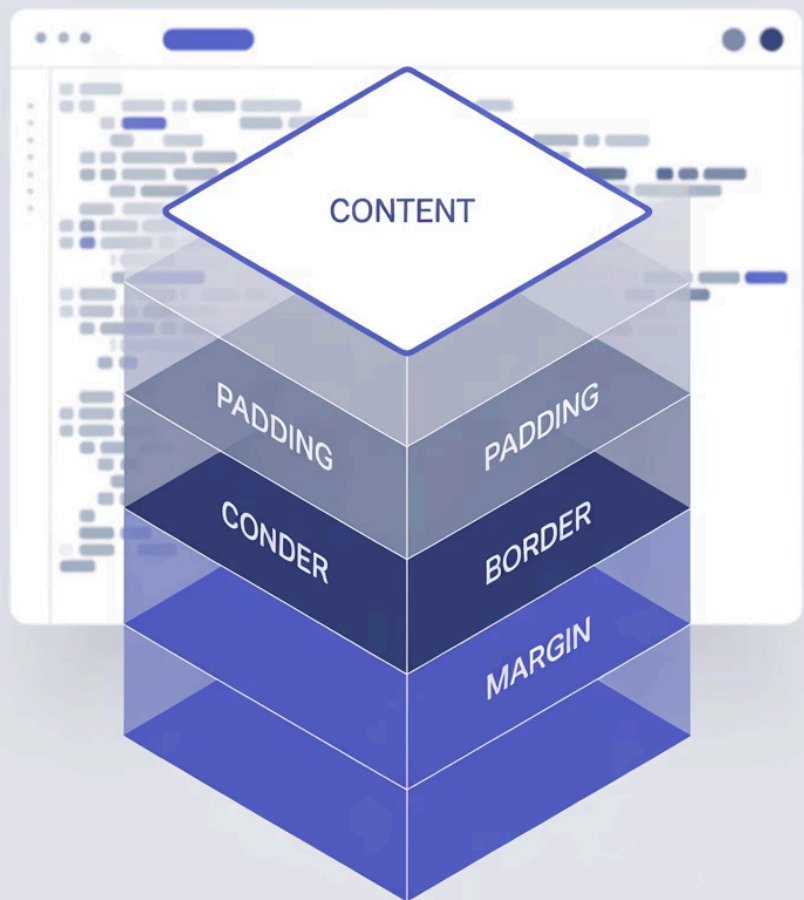


Box Model



Box Model e Display em CSS3

Um guia completo sobre os fundamentos do layout web moderno, explorando como elementos são estruturados e exibidos no navegador.

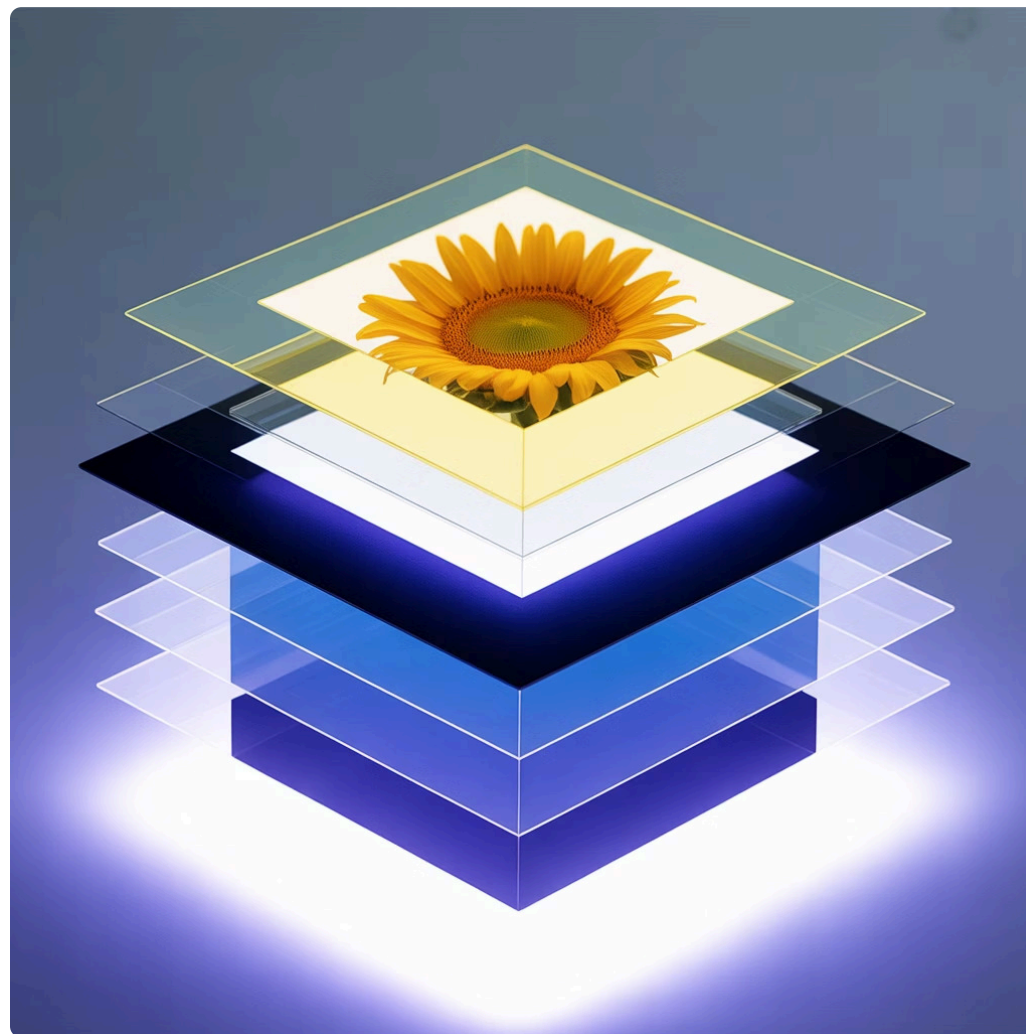


por **Professor Gilmar dos Santos Freire**

O que é o Box Model?

O Box Model é o conceito fundamental que define como cada elemento HTML é renderizado como uma caixa retangular no navegador. Compreender este modelo é essencial para criar layouts precisos e responsivos.

Cada elemento possui quatro áreas distintas: conteúdo, padding, border e margin. Estas camadas trabalham juntas para determinar o espaço total ocupado pelo elemento na página.



Anatomia do Box Model

Content

O conteúdo real do elemento - texto, imagens ou outros elementos aninhados.

Propriedades: `width`, `height`

Padding

Espaço transparente entre o conteúdo e a borda do elemento.

Propriedades: `padding-top`, `padding-right`, `padding-bottom`, `padding-left`

Border

A linha que contorna o elemento, podendo ter diferentes estilos e espessuras.

Propriedades: `border-width`, `border-style`, `border-color`

Margin

Espaço transparente externo que separa o elemento de outros elementos adjacentes.

Propriedades: `margin-top`, `margin-right`, `margin-bottom`, `margin-left`

Box-Sizing: Controlando o Cálculo

content-box (padrão)

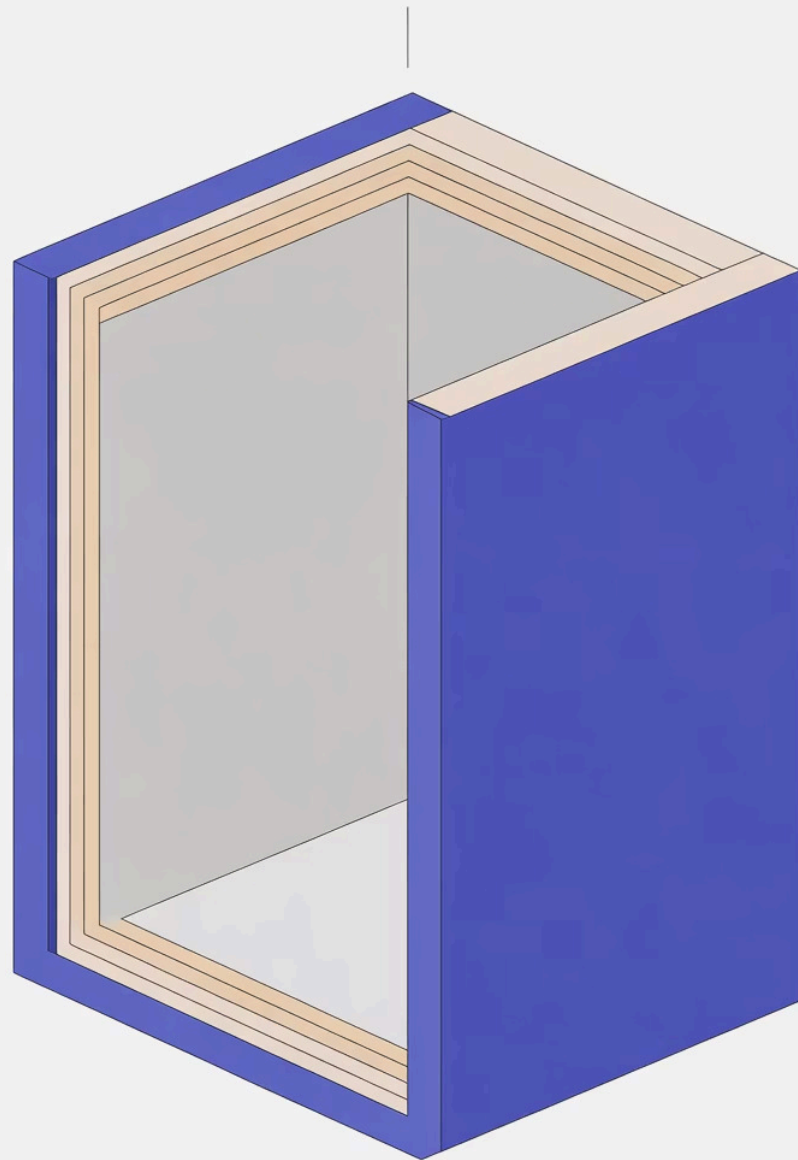
Width e height aplicam-se apenas ao conteúdo. Padding e border são adicionados ao tamanho total.

```
box-sizing: content-box;  
width: 200px;  
padding: 20px;  
/* Largura total = 240px */
```

border-box

Width e height incluem conteúdo, padding e border. Mais previsível para layouts responsivos.

```
box-sizing: border-box;  
width: 200px;  
padding: 20px;  
/* Largura total = 200px */
```



Content-Box

Aditive side

Border-Box,

inclusive width heigh,

Propriedades de Dimensão

Largura (Width)

- `width: auto` - Largura automática
- `width: 300px` - Valor absoluto
- `width: 50%` - Valor relativo
- `min-width: 200px` - Largura mínima
- `max-width: 800px` - Largura máxima

Altura (Height)

- `height: auto` - Altura automática
- `height: 400px` - Valor absoluto
- `height: 100vh` - Viewport height
- `min-height: 150px` - Altura mínima
- `max-height: 600px` - Altura máxima

Cool's Or, Deltij

PixelBloom Design dear Agency

We're anling of bearchneods Oomlat
as zennodrevacim, esoling,
beaglio.

Web Design

Lores teune colurs dntessuet
onidhnoitstibentitbeciree
oluting vntestioe tuit.

Learn more

Ui/UX Consulting

Lairrex taune onlas slottssuet
onidhnoitstibentitbeciree
oluting vntestioe tuit.

Learn more

Brand Identity

Lores teune onlas slottstuet
onidhnoitstibentitbeciree
oluting vntestioe tuit.

Learn more

Trabalhando com Padding

O padding cria espaço interno no elemento, sendo crucial para a legibilidade e estética. Pode ser definido individualmente para cada lado ou usando shorthand.

Sintaxe Individual

```
padding-top: 10px;  
padding-right: 15px;  
padding-bottom: 20px;  
padding-left: 25px;
```

Shorthand - 4 valores

```
/* top | right | bottom | left */  
padding: 10px 15px 20px 25px;
```

Shorthand - 2 valores

```
/* vertical | horizontal */  
padding: 20px 40px;
```

Shorthand - 1 valor

```
/* todos os lados iguais */  
padding: 15px;
```

Configurando Bordas

Propriedade	Valores Possíveis	Exemplo
border-style	solid, dashed, dotted, double, groove, ridge, inset, outset, none	border-style: solid;
border-width	thin, medium, thick, ou valores em px	border-width: 2px;
border-color	Nomes, hex, rgb, rgba, hsl	border-color: #3498db;
border (shorthand)	width style color	border: 1px solid black;
border-radius	Valores em px, em, %	border-radius: 10px;

Gerenciando Margins

As margins controlam o espaço externo entre elementos. Um conceito importante é o **margin collapse**, onde margins verticais adjacentes se combinam.

1

Margin Collapse

Quando dois elementos verticais possuem margins, apenas a maior é aplicada, não a soma das duas.

```
elemento1 { margin-bottom: 20px; }  
elemento2 { margin-top: 30px; }  
/* Espaço resultante: 30px (não 50px) */
```

2

Centralizando Horizontalmente

Use `margin: 0 auto` para centralizar elementos de bloco com largura definida.

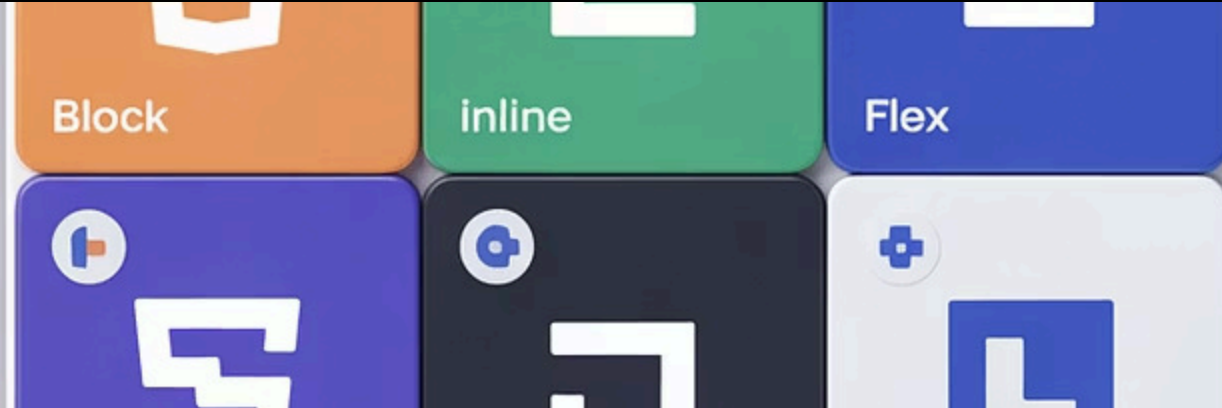
```
.container {  
  width: 800px;  
  margin: 0 auto;  
}
```

3

Valores Negativos

Margins podem ter valores negativos, permitindo sobreposição e efeitos visuais únicos.

```
.overlap {  
  margin-top: -20px;  
}
```

Introdução à Propriedade Display

A propriedade `display` define como um elemento é renderizado e como interage com outros elementos. É fundamental para controlar o layout e o fluxo de elementos na página.

Cada elemento HTML possui um valor de display padrão (block ou inline), mas podemos alterá-lo para criar layouts mais complexos e flexíveis.

Display Block vs Inline

display: block

Elementos ocupam toda a largura disponível e quebram linha.

- Aceita propriedades de width/height
- Inicia em nova linha
- Exemplos: div, h1-h6, p, section

```
.bloco {  
  display: block;  
  width: 100%;  
  height: 50px;  
}
```

display: inline

Elementos fluem horizontalmente, lado a lado, respeitando o fluxo do texto.

- Ignora propriedades width/height
- Não quebra linha
- Exemplos: span, a, strong, em

```
.inline {  
  display: inline;  
  /* width/height ignorados */  
}
```

Display: Inline-Block

O `display: inline-block` combina características de elementos `block` e `inline`, oferecendo flexibilidade única para layouts.

Características:

- Flui horizontalmente como `inline`
- Aceita `width/height` como `block`
- Ideal para botões e cards lado a lado
- Mantém espaçamento natural entre elementos



```
.card {  
  display: inline-block;  
  width: 200px;  
  height: 150px;  
  margin: 10px;  
  vertical-align: top;  
}
```

Display: None vs Visibility

display: none

Remove completamente o elemento do fluxo da página. O elemento não ocupa espaço e não é renderizado.

```
.oculto {  
  display: none;  
  /* Elemento inexistente */  
}
```

visibility: hidden

Torna o elemento invisível mas mantém seu espaço no layout. O elemento ainda influencia o posicionamento de outros elementos.

```
.invisivel {  
  visibility: hidden;  
  /* Espaço preservado */  
}
```



Valores Modernos de Display



display: flex

Cria um contêiner flexível para alinhamento e distribuição dinâmica de elementos filhos. Ideal para layouts unidimensionais (linha ou coluna).

```
.flex-container {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```



display: grid

Estabelece um sistema de grid bidimensional com controle preciso sobre linhas e colunas. Perfeito para layouts complexos.

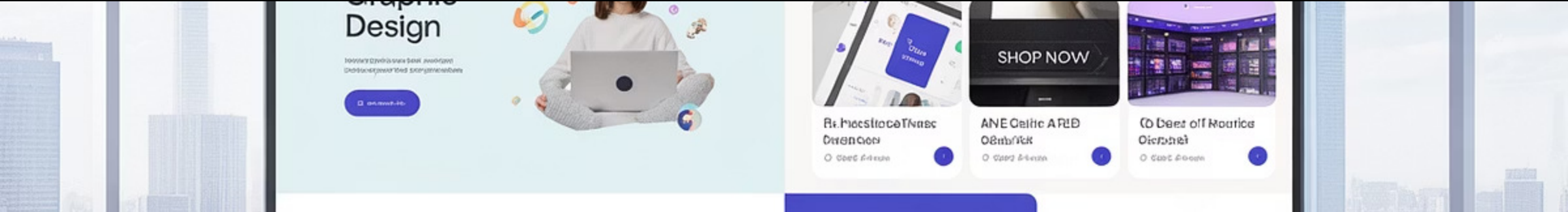
```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 2fr;  
  gap: 20px;  
}
```



display: table

Faz elementos se comportarem como tabelas HTML, útil para alinhamento vertical e layouts tabulares sem usar table tags.

```
.table-layout {  
  display: table;  
  width: 100%;  
}
```



Casos Práticos e Exemplos

Vamos ver exemplos práticos de como combinar Box Model e Display para criar layouts eficientes.

Card Component

```
.card {
  display: inline-block;
  width: 300px;
  padding: 20px;
  margin: 15px;
  border: 1px solid #ddd;
  border-radius: 8px;
  box-sizing: border-box;
}
```

Navigation Bar

```
.nav {
  display: flex;
  padding: 0 20px;
  border-bottom: 2px solid #333;
}
.nav-item {
  display: block;
  padding: 15px 20px;
  margin-right: 10px;
}
```

Responsive Container

```
.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 20px;
  box-sizing: border-box;
}
@media (max-width: 768px) {
  .container { padding: 0 10px; }
}
```

Resumo e Próximos Passos



Box Model Dominado

Content, padding, border e margin trabalham juntos. Use `box-sizing: border-box` para cálculos previsíveis.



Display Flexível

Block, inline, inline-block, flex e grid oferecem controle total sobre o layout dos elementos.



Próximos Estudos

Aprofunde-se em Flexbox e CSS Grid para layouts avançados. Pratique com projetos reais para consolidar o conhecimento.

Domine estes fundamentos e você terá a base sólida necessária para criar qualquer layout web moderno e responsivo!